

2023年度 プログラミングⅣ 第13週

担当教員: 小林 邦和

担当TA: 中村(M2), 柴田(B4)

前回の授業の復習

グローバル変数

プログラム開始

プログラム終了



ローカル変数(動的)

関数呼び出し開始

呼び出し終了

呼び出し開始

呼び出し終了

プログラム開始

プログラム終了



ローカル変数(静的)

プログラム開始

プログラム終了



- 関数の呼び出しにおいて、**自分自身を呼び出す**こと

例

```
int func(int x)
{
    ...
    return func(x-1);
}
```

関数func()の中で、
自分自身の関数func()を
呼び出している

関数の再帰的呼び出し

- 関数の引数: **個数に制限なし**
- 関数の戻り値: **スカラーのみ**
- 次のような複素数 s で, x と y を引数として与える

$$s = x + iy$$

- その極座標表現 r と θ の2つの値を戻り値として返す関数は設計できない

$$\begin{cases} r = \sqrt{x^2 + y^2} \\ \theta = \tan^{-1}(y/x) \end{cases}$$

解決策(関数処理で複数の値を返す)

529

&rは変数rのアドレス使用
例: scanf("%d", &n);

- 参照渡しの利用
- 関数: xy_rt(x, y, &r, &t);
- 複素変数のxとyは, 値渡しで関数へ渡す

$$s = x + iy$$

- 極座標表現のrと θ は, 参照渡しで関数へ渡す

$$\begin{cases} r = \sqrt{x^2 + y^2} \\ \theta = \tan^{-1}(y/x) \end{cases}$$

• 値渡しの場合

- ✓ 呼び出す側: **実引数の値渡し**
- ✓ 呼び出される側: **仮引数のメモリ領域(104番地)を確保**

• 参照渡しの場合

- ✓ 呼び出す側: **実引数のアドレス(101番地)を渡す**
- ✓ 呼び出される側: **仮引数のアドレスは、実引数と同一**

アドレス	メモリ領域
...	...
100	
101	実引数
102	
103	
104	仮引数
105	
...	...

- 引数が**スカラー変数**の場合

- ✓ 実引数と仮引数は、それぞれ**異なるメモリ領域**に確保 → 物理的に**非同一**

- 引数が**配列変数**の場合

- ✓ **同じメモリ領域**に確保
- ✓ 配列変数は、関数内で**実引数(配列)**の値を操作可
→ 物理的に**同一**

→ 引数が**値渡し**しか、**参照渡し**しかの違い

スカラー引数の場合

532

仮引数

```
void func(int x)
```

```
{
```

```
x = 10;
```

```
}
```

```
int main(void)
```

```
{
```

```
int n=1;
```

```
func(n);
```

```
printf("n=%d¥n", n);
```

```
return 0;
```

```
}
```

仮引数xに10を
代入しても、実引
数nは1のまま

実引数nを関数
func()へ渡す

アドレス	メモリ領域
...	...
100	
101 (= &n)	1 (= n)
...	
103	
104 (= &x)	10 (= x)
...	...

配列引数の場合

仮引数はポインタでも可

533

```
void func( int *x)
```

配列の仮引数

```
{
```

```
    x[0] = 10;
```

```
}
```

```
int main(void)
```

```
{
```

```
    int n[5]={ 1, 2, 3, 4, 5};
```

```
    func( n );
```

```
    printf("n[0]=%d¥n", n[0]);
```

```
    return 0;
```

```
}
```

仮引数と実引数は同じメモリ領域のため、
x[0], つまりn[0]に
10が代入される

実引数nを関数func()
へ引き渡す

アドレス	メモリ領域
...	...
100	
101 (=&n[0]) (=&x[0])	1 (=n[0]) → 10 (=x[0])
...	
103	
...	...

第8回 配列を用いたプログラミング演習

第9回 関数

第10回 関数を用いたプログラミング演習

第11回 関数の再帰的呼び出し

第12回 関数の再帰的呼び出しを用いたプログラミング演習

第13回 デバッグ

第14回 デバッグの演習

今週の授業内容 デバッグ (教科書に記載がない内容)

① デバッグ

プログラムのバグの概念とバグ取りの方法

- 由来

英語のdebug

- 意味

コンピュータプログラムから間違い(バグ)を取り除く

- debugの別の意味

虫を取る

→デバッグのことを虫取りとも言う

- **文法的**なバグ

誤記(タイプミス)など

→ **コンパイルエラー**で**検知**できる場合が多い

- **論理的**なバグ

アルゴリズム上のミス, 間違った変数型の宣言など

→ **コンパイルエラー**が出ないので発見が難しい

→ **デバッガ**や統合開発環境の利用

コンパイルエラーが出ない

→ **文法的なバグがない**

コンパイルの際に、**-g**と**-O0**を付ける

```
gcc -g -O0 -o sample sample.c
```

※**-g**は実行ファイルに**デバッグ情報を付加**する

※**-O0**はプログラムを**最適化しない**という意味

-(マイナス)**O**(英大文字のオー)**0**(数字のゼロ)

ターミナルのコマンドラインから以下を入力

```
gdb ./sample
```

※./sampleは実行ファイル名

コマンド	意味
run	実行
break 関数名or行番号	ブレイクポイント(BP)の設定
break 関数名 if 条件式	条件付きBPの設定
info breakpoints	BPの表示
delete BP番号	BPの削除

※赤字で省略可

コマンド	意味
d isable BP番号	BPの無効化
e nable BP番号	BPの有効化
n ext	次の行へ(関数内の処理は見ない)
s tep	次の行へ(関数内の処理まで見る)
c ontinue	継続実行(次のBPまで)

コマンド	意味
w atch 変数名	ウォッチポイントの指定
p rint 変数名	変数名の値の表示
p rint 変数=値	変数へ値の代入
l ist	ソースの表示(デフォルト10行)
h elp コマンド	ヘルプの表示
q uit	終了

1. emacsを起動する
2. プログラムファイルを開く
3. コマンド入力モードへ移行する
ESC→xの順にキーを打つ
4. linum-modeと入力する

入手手順

1. ユニパからsample_code.zipをダウンロード
2. ターミナルを起動し, ダウンロードしたフォルダへ移動し, `unzip sample_code.zip`と打つ
3. Lesson13というフォルダ下に以下のファイルが解凍

ファイルリスト

- sample1300.c
- sample1301.c
- sample1302.c
- sample1303.c

gdbの動作確認

```
/* sample1300.c: 2つの対角線の値(整数)を入力し, ひし形の面積を求める */  
#include <stdio.h>  
  
int main(void)  
{  
    int    taikaku1, taikaku2;  
    double menseki;  
    puts("ひし形の面積を計算します");  
    printf("対角線の1つ目の値の入力: ");  
    scanf("%d", &taikaku1);  
    printf("対角線の2つ目の値の入力: ");  
    scanf("%d", &taikaku2);  
    menseki = taikaku1 * taikaku2 / 2.0;  
    printf("対角線%dと%dのひし形の面積は%.1fです\n", taikaku1, taikaku2, menseki);  
    return 0;  
}
```

- デバッグオプションを付してコンパイル

```
gcc -g -O0 -o sample1300 sample1300.c
```

- デバッガの起動

```
gdb ./sample1300
```

- ブ레이크ポイント (BP) の設定
(gdb) b main
- BPまで実行
(gdb) r
- 1行ずつ実行
(gdb) n
- ソースの表示
(gdb) l
- 入力した変数の値の確認
(gdb) p taikaku1
(gdb) p taikaku2

- mensekiの計算前に, taikaku I の値の変更
(gdb) p taikaku I = 10
- 変更した変数の値の確認
(gdb) p taikaku I
- 計算結果(menseki)の確認
(gdb) p menseki
- 継続実行
(gdb) c
- デバッガの終了
(gdb) q

演習 I

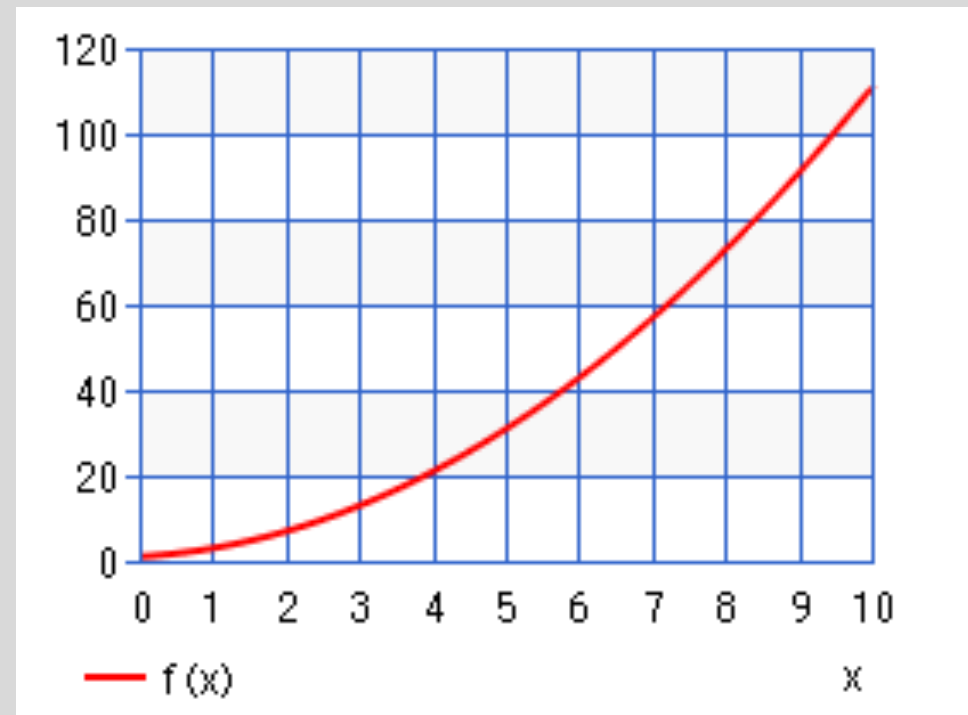
2次関数の計算[1/2]

553

```
/* sample1301.c */
#include <stdio.h>
#include <stdlib.h>
#define square(x)    ((x) * (x))
#define NMAX 100
#define DEFAULTMAX 5
int func_a(int x)
{
    int y;

    y = square(x) + x + 1;

    return y;
}
```



2次関数の計算[2/2]

554

```
int main(void)
{
    int x, y[NMAX], maxnum;

    puts("maxnum?");
    if (scanf("%d", &maxnum) != 1)
        maxnum = DEFAULTMAX;
    for (x = 0; x <= maxnum; x++)
        y[x] = func_a(x);
    for (x = 0; x <= maxnum; x++)
        printf("x=%d, y=%d¥n", x, y[x]);

    return 0;
}
```

- ブレイクポイント (BP) の設定
(gdb) b main
(gdb) b func_a
- BPの確認
(gdb) i b
※mainが1つ目, func_aが2つ目のBP
- 最初のBP(main)まで実行
(gdb) r
- 1行ずつ実行
(gdb) n

- 関数内も1行ずつ実行
(gdb) s
- 2つ目のBP(func_a)を無効化
(gdb) dis 2
- BPの確認
(gdb) i b
※2つ目のBPが無効
- 次のBP(main)まで継続実行
(gdb) c
- デバッガの終了
(gdb) q

演習2

コンビネーションの計算[1/2]

558

```
/* sample1302.c */
#include <stdio.h>
#include <stdlib.h>

/* --- 階乗値を返す --- */
int factorial(int n)
{
    if (n > 0)
        return n * factorial(n - 1);
    else
        return 1;
}
```

```
int main(void)
{
    int    n, r, ncr;

    scanf("%d %d", &n, &r);

    ncr = factorial(n) / (factorial(r) * factorial(n-
r));

    printf("n=%d, r=%d, nCr=%ld¥n", n, r, ncr);

    return 0;
}
```

- ブレイクポイントの設定
(gdb) b factorial if n==0
(gdb) b main
※factorialのBPはn==0のときのみ有効
- BPの確認
(gdb) i b
※factorialが1つ目, mainが2つ目のBP
- 最初のBP(main)まで実行
(gdb) r
- 1行ずつ実行
(gdb) n

- 関数内も1行ずつ実行
(gdb) s
- ソースの表示
(gdb) l
- 条件を満たす次のBPまで継続実行
(gdb) c
- デバッガの終了
(gdb) q

演習3

配列の受け渡し[1/2]

563

```
/* sample1303.c */
#include <stdio.h>

/*--- 要素数nの配列vの要素に0を代入 ---*/
void set_zero(int v[], int n)
{
    int i;

    for (i = 0; i < n; i++)
        v[i] = 0;
}

/*--- 要素数nの配列vの全要素を表示して改行 ---*/
void print_array(const int v[], int n)
{
    int i;

    printf("{ ");
    for (i = 0; i < n; i++)
        printf("%d ", v[i]);
    printf("}");
}
```

配列の受け渡し[2/2]

564

```
int main(void)
{
    int ary1[] = {1, 2, 3, 4, 5};
    int ary2[] = {3, 2, 1};

    printf("ary1 = "); print_array(ary1, 5); putchar('\n');
    printf("ary2 = "); print_array(ary2, 3); putchar('\n');

    set_zero(ary1, 5);          /* 配列ary1の全要素に0を代入 */
    set_zero(ary2, 3);          /* 配列ary2の全要素に0を代入 */

    printf("両配列の全要素に0を代入しました。\n");
    printf("ary1 = "); print_array(ary1, 5); putchar('\n');
    printf("ary2 = "); print_array(ary2, 3); putchar('\n');

    return 0;
}
```

- ブ레이크ポイントの設定

(gdb) b main

(gdb) b set_zero

(gdb) b print_array

- ブ레이크ポイントの確認

(gdb) i b

※mainが1つ目, set_zeroが2つ目, print_arrayが3つ目のBP

- 1つ目のBP(main)まで実行

(gdb) r

- 1行ずつ実行
(gdb) n
- 次のBP(print_array)まで継続実行
(gdb) c
- 配列値の変更(再代入)
(gdb) p v[0]=1
(gdb) p v[1]=2
(gdb) p v[2]=3
- 次のBP(print_array)まで継続実行
(gdb) c
- デバッガの終了
(gdb) q

期末試験と評価方法

期末試験

日時

2月1日(木) 2限

会場

S201

通常の演習室6で
はないので注意

期末試験では、授業・演習を通して、C言語に関する下記**3つのポイント**が修得できているかを確認します

1. **用語・概念**を説明できる
2. **プログラム**を読み書きできる
3. **式の計算, 繰り返し処理, 条件分岐, 配列, 関数(再帰含む)**を扱うことができる

- 講義ノートや参考書などの**持ち込み禁止**
- **不正行為厳禁**
- 試験時間中, インターネットへ接続できる機器の**使用**だけでなく, **身につけること**や**机上に置くこと**も, 原則不正行為とみなす
- 不正行為は, 当該期に履修登録しているすべての科目の成績が**D評価**となる

下記**3項目**の総合評価(シラバス記載の通り)

- 期末試験【**60%**】
- 演習課題【**20%**】
- 授業態度・小テスト【**20%**】

欠席6回以上

※出席が**2/3に満たない**場合は, 期末試験の受験資格なし(欠格条件)

※演習課題・小テストの提出が**2/3に満たない**場合は, 期末試験の成績に関わらず**不合格**